

SISMID 2026 Spatial Statistics Waller Point Process 2: Dental Intensities

Lance A. Waller

7/19/2026

Spatial point processes: Intensities and archaeology

Grave dental data from Richard Wright in Australia, Emeritus Professor, School of Archaeology, University of Sydney NSW 2006, Australia

We'll need two libraries for this...there are fancier options now, but these will do the trick.

```
library(splancs)
```

```
## Loading required package: sp
```

```
##
```

```
## Spatial Point Pattern Analysis Code in S-Plus
```

```
##
```

```
## Version 2 - Spatial and Space-Time analysis
```

```
library(KernSmooth) # Matt Wand's kernel smoothing techniques
```

```
## KernSmooth 2.23 loaded
```

```
## Copyright M. P. Wand 1997-2009
```

```
library(here)
```

```
## here() starts at /Users/lwaller/Library/CloudStorage/OneDrive-Emory/meetings/SISMID.2026/lectures/Le
```

```
library(RColorBrewer)
```

Set working directory and read in the data (dental.reduced.dat) and a polygon outline file (created earlier...commented code below shows how to create your own in splancs).

```
#####
```

```
# Set path to data sets, etc.
```

```
#####
```

```
# Set my working directory (Lance's here for example)
```

```
#setwd("~/OneDrive - Emory University/meetings/SISMID.2021/SISMID.2021.Waller.Rcode")
```

```
***HERE***
```

```
#path = "/Users/lwaller/Library/CloudStorage/OneDrive-Emory/meetings/SISMID.2024/SISMID_2024_spatial_st
```

```
# Read in data with south region removed
```

```
dental <- scan(here("data", "dental.reduced.dat"),list(lab=0,aff=0,x=0,y=0))
```

```
****HERE****
```

```
#dental <- scan(paste(path,"data/dental.reduced.dat",sep=""),list(lab=0,aff=0,x=0,y=0))
```

```
# read in previously defined polygon boundary
mypoly <- read.table(here("data", "mypoly.dat"))
#mypoly <- read.table(paste(path, "data/mypoly.dat", sep=""))
```

Now to set up the data and make preliminary plots (with some adjustments to have square plots).

```
# Set points as spatial point process data for splancs
dental.p <- as.points(dental$x,dental$y)
dentcas.p <- as.points(dental$x[dental$aff==1],dental$y[dental$aff==1])
dentcon.p <- as.points(dental$x[dental$aff==0],dental$y[dental$aff==0])

par(pty="s")

# dental$y has a bigger range so this tries to add half of the
# extra to each side

extra <- ( diff(range(dental$y))-diff(range(dental$x)) )/2

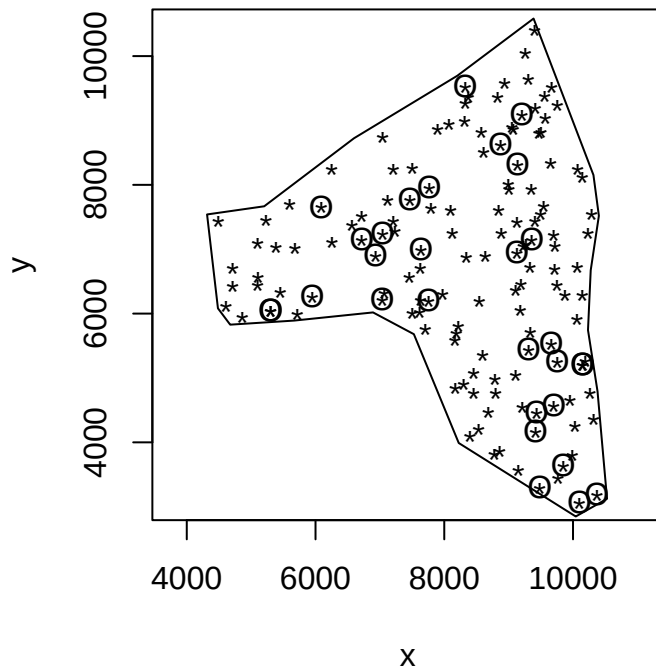
plot(dental.p,pch="*",xlim=c(min(dental$x)-extra,max(dental$x)+extra),
      ylim=range(dental$y),
      xlab="x",ylab="y")
title("Grave locations (*=grave, 0=affected)")

points(dentcas.p,pch="0")

## Interactively define polygon -- run this function and click
# in the plot where you want the vertices of the polygon
# to be. ("Connect the dots" around the points, then
# click right button to select 'done').
# mypoly <- getpoly()

# Add bounding polygon to the plot.
polygon(mypoly)
```

Grave locations (*=grave, O=affected)



The next section considers how to *compare* two intensity functions. Do the peaks and valleys line up?

In spatial epidemiology we are interested in comparing the intensity of cases to the intensity of controls. Where do these differ? Where are cases more likely than controls, or vice versa?

Since each spatial intensity (the expected number of events at a certain location) is proportional to the spatial density (the probability of observing an event at a certain location), if we take the ratio of two intensities, this corresponds to the *spatial relative risk* of observing a case event versus a control event. Mapping the spatial relative risk function gives a sense of where cases are more likely than controls. (Kelsall and Diggle, 1995 *Statistics in Medicine*).

The “sparr” package does a lot of this automatically, but to see how it works, here is the “classic” code used to generate this for the Waller and Gotway textbook.

First, we get the case intensity and the control intensity *with the same bandwidth* (so the two surfaces have the same flexibility and we’re not finding noise over signal). The code below uses the “KernSmooth” package to get kernel smoothed density estimates (proportional to intensity estimates).

We will define the grid of points at which we calculate the estimate

We’re setting the bandwidth to 700 distance units. Calculating the density estimates and plotting both.

```
#####
# Kelsall and Diggle for bandwidth = 700
#pdf("dental.intensities.700.pdf",width=11,height=8)
par(mfrow=c(1,2),pty="s")

casbandw <- 700
conbandw <- 700

# define number of grid points in a row and in a column (same number for now)
grid.number <- 40
```

```

casdens <- bkde2D(dentcas.p, casbandw, gridsize=c(grid.number,grid.number),
  range.x=list( c( min(dentcas.p[,1],dentcon.p[,1])-1.5*casbandw,
    max(dentcas.p[,1],dentcon.p[,1])+1.5*casbandw),
    c( min(dentcas.p[,2],dentcon.p[,2])-1.5*casbandw,
    max(dentcas.p[,2],dentcon.p[,2])+1.5*casbandw) ) )

condens <- bkde2D(dentcon.p, conbandw, gridsize=c(grid.number,grid.number),
  range.x=list( c( min(dentcas.p[,1],dentcon.p[,1])-1.5*casbandw,
    max(dentcas.p[,1],dentcon.p[,1])+1.5*casbandw),
    c( min(dentcas.p[,2],dentcon.p[,2])-1.5*casbandw,
    max(dentcas.p[,2],dentcon.p[,2])+1.5*casbandw) ) )

# use splancs to identify points in mypoly polygon
# and set density estimates to NA outside of bounding polygon

gridlocs.y <- rep(casdens$x2[1],grid.number)
for (i in 2:grid.number) {
  gridlocs.y <- c(gridlocs.y,rep(casdens$x2[i],grid.number))
}
gridlocs.x <- rep(casdens$x1,grid.number)

gridlocs <- as.points(gridlocs.x,gridlocs.y)
inside <- inout(gridlocs,mypoly)

## Warning in as.points(poly): List/data.frame components should be named x and y

# This will set any grid point that is "not inside" (!inside) to NA so it won't show up
# in the plot.
casdens$fhat[!inside] <- NA
condens$fhat[!inside] <- NA

# 'persp' will make a fishnet plot of the surface
#persp(casdens$x1,casdens$x2,casdens$fhat,theta=30,phi=45,xlab="u",
#      ylab="v", zlab="Intensity",cex=2)
#title("Estimated intensity function",cex.main=1.75)

# 'plot' followed by 'contour' will plot the dots, then add (add=T) the contours to the plot
plot(dentcas.p,xlim=c(min(dental$x)-extra,max(dental$x)+extra),
  ylim=range(dental$y),
  xlab="u",ylab="v",cex.axis=1.25,cex.lab=1.5)
polygon(mypoly)
title("Affected sites",cex.main=1.5)
contour(casdens$x1,casdens$x2,casdens$fhat,add=T,
  vfont=c("sans serif","bold"),
  levels=c(0,0.00000001,0.00000002,0.00000003,0.00000004,0.00000005),
  drawlabels=F)

# if you want to add labels to the contours, replace the last ')' with a comma
#      labels=c(0,0.00000001,0.00000002,0.00000003,0.00000004,0.00000005),
#      labcex=0.8)

# now for the control surface
#persp(condens$x1,condens$x2,condens$fhat,theta=30,phi=45,xlab="u",
#      ylab="v", zlab="Intensity",cex=2)
#title("Estimated intensity function",cex.main=1.75)

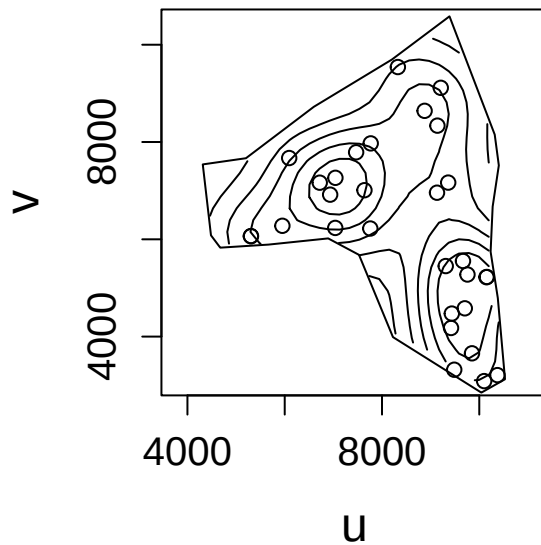
```

```

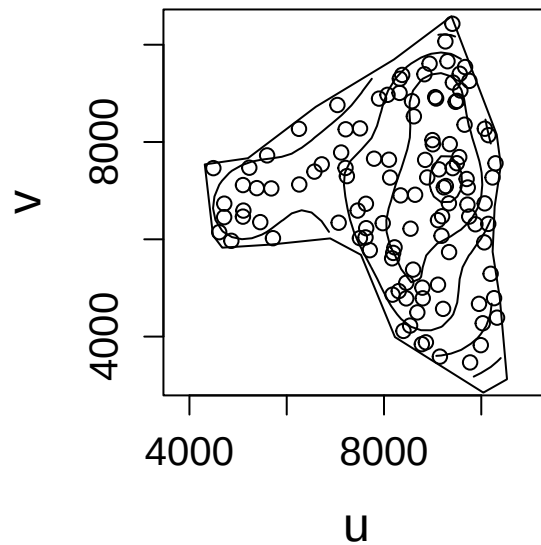
plot(dentcon.p,xlim=c(min(dental$x)-extra,max(dental$x)+extra),
     ylim=range(dental$y),
     xlab="u",ylab="v",cex.axis=1.25,cex.lab=1.5)
polygon(mypoly)
title("Non-affected sites",cex.main=1.5)
contour(condens$x1,condens$x2,condens$fhat,add=T,
        vfont=c("sans serif","bold"),
        levels=c(0,0.00000001,0.00000002,0.00000003,0.00000004,0.00000005),
        drawlabels=F)

```

Affected sites



Non-affected sites



```

#      labels=c(0,0.00000001,0.00000002,0.00000003,0.00000004,0.00000005),
#      labcex=0.8)
#dev.off()

```

Now we'll calculate the relative risk surface by finding the ratio of the density estimates at the same grid points.

```

#####
# Kelsall and Diggle's ratio of intensity surfaces using KernSmooth

# Define the relative risk surface as the
# point-by-point ratio of case density to control density
r.ks <- log(casdens$fhat/condens$fhat)
# set any 'divide by zero' problems to 'NA'
r.ks[!is.finite(r.ks)] <- NA

# leave out the grid points outside the polygon
r.ks[!inside] <- NA

```

We'll be doing a Monte Carlo test at each grid point.

So we need a matrix of all of the test statistics (one for each grid point).

```

# calculate overall test statistic (divide each value of r.ks by area
# of grid squares

```

```
test.stat.obs <- sum( (r.ks[!is.na(r.ks)]/(diff(casdens$x1)[1]*diff(casdens$x2)[1]))^2 )
```

Now to plot the relative risk surface.

```
#####
# Plot relative risk surface

#pdf(paste("dentalKD.tol700.KS.pdf",width=11,height=8)
  par(mfrow=c(1,2),pty="s")

  persp(casdens$x1,casdens$x2,r.ks,theta=-25,phi=45,
        xlab="u",ylab="v",zlab="log relative risk",zlim=c(-3,3))

  title("Log relative risk surface")

# dental$y has a bigger range so this tries to add half of the
# extra to each side
extra <- ( diff(range(dental$y))-diff(range(dental$x)) )/2

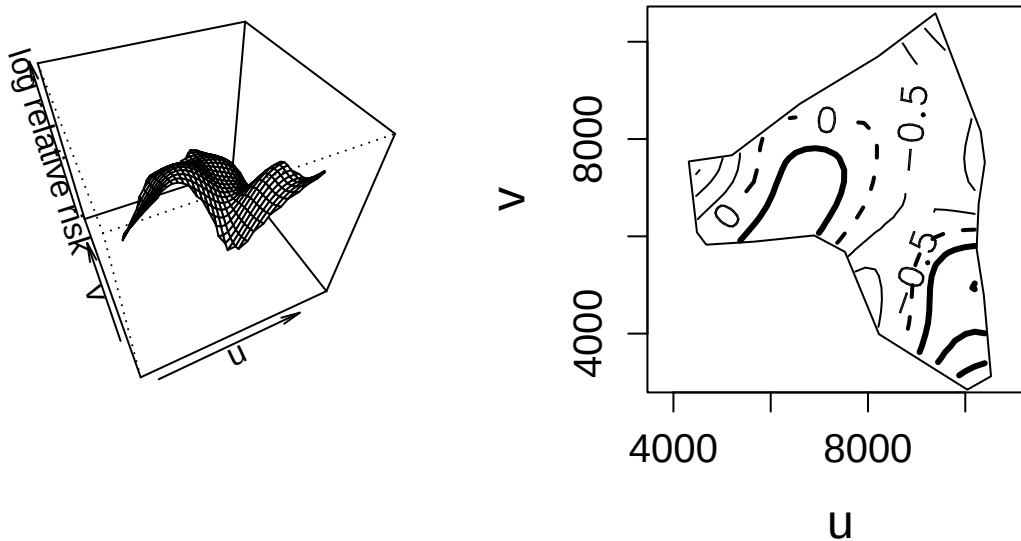
plot(dentcon.p,pch=" ",xlim=c(min(dental$x)-extra,max(dental$x)+extra),
     ylim=range(dental$y),xlab="u",ylab="v",cex.axis=1.25,cex.lab=1.5)
contour(casdens$x1,casdens$x2,r.ks,levels=c(-2,-1.5,-1,-0.5,0,0.5,1,1.5,2),
       lwd=c(1,1,1,1,2,3,3,3,3),
       lty=c(1,1,1,1,2,1,1,1,1),
       vfont=c("sans serif","bold"),
       labcex=1.25,add=T )
polygon(mypoly)

# you can add the points if you want to.  'pch' = 'plot character'.
# pch = 16 is a filled circle
# pch = 1 is an open circle
#   points(dentcas.p,pch=16)
#   points(dentcon.p,pch=1)

#Add a title pasting the bandwidth
  title(paste("Bandwidth =",casbandw))
```

Log relative risk surface

Bandwidth = 700



```
#dev.off()
```

Now we want to figure out where the relative risk surface is more (or less) than we would expect under the null hypothesis where the local relative risk is 1.0 (the null hypothesis).

The steps are:

- For each simulation, select 30 locations from the set of all locations to be “cases”. (Done by sampling 30 indices from the set of 1:(total number of location), and labeling these “cases” and the others “controls”.
- Estimate the case and control intensities, take the ratios and save the values for each grid point.
- After the simulations, we will have (number of simulations) values of the relative risk *at each grid point* under the null distribution.
- We also have the relative risk value from the data *at the same grid points*.
- We use the quantile function to find the 95th and 5th quantiles of the null simulation values *at each grid point*.
- If the data-based relative risk value is > the 95th percentile, write a “+” *at that grid location*.
- If the data-based relative risk value is < the 5th percentile, write a “-” *at that grid location*.

We’ll end up with a map showing areas significantly higher (+’s) and significantly lower (-’s) than the null relative risk. These are hot and cold spots, respectively.

```
#####  
# Now for Kelsall and Diggle tolerance surfaces (identify the grid points that  
# are higher than the 97.5th percentile of Monte Carlo surfaces with "+" and  
# those that are lower than the 2.5th percentile with "-").  
#####  
  
# Define the bandwidth  
bandw <- 700  
  
# Define the number of simulations  
sim <- 999
```

```

# 'ind' is an index that goes from 1 to the number of points
ind <- 1:length(dental$x)

# 'rrandmat' is a matrix of zeros with 'sim' rows and a column for each grid point
rrandmat <- matrix(0,sim,grid.number*grid.number)

# 'test.stat.sim' is a vector of zeros to hold the statistic values from each simulation
test.stat.sim <- rep(0,sim)

# Here is the simulation loop
for (i in 1:sim) {

# this is the random label assignment...take a sample of size
# (how many cases are there?) = length of the vector of x-coordinates of the cases
# from 'ind' (the index)
randind <- sample(ind, length(dental$x[dental$aff==1]))

# define the random labelling cases
dentrandx <- dental$x[randind]
dentrandy <- dental$y[randind]
dentrand.cas.p <- as.points(dentrandx,dentrandy)
# define the random labelling controls
dentrandx.0 <- dental$x[-randind]
dentrandy.0 <- dental$y[-randind]
dentrand.con.p <- as.points(dentrandx.0,dentrandy.0)

# Get the random labelling case density estimate
frand <- bkde2D(dentrand.cas.p, casbandw, gridsize=c(grid.number,grid.number),
               range.x=list( c( min(dentcas.p[,1],dentcon.p[,1])-1.5*casbandw,
                                max(dentcas.p[,1],dentcon.p[,1])+1.5*casbandw),
                             c( min(dentcas.p[,2],dentcon.p[,2])-1.5*casbandw,
                                max(dentcas.p[,2],dentcon.p[,2])+1.5*casbandw) ) )

# Get the random labelling control density estimate
grand <- bkde2D(dentrand.con.p, conbandw, gridsize=c(grid.number,grid.number),
               range.x=list( c( min(dentcas.p[,1],dentcon.p[,1])-1.5*casbandw,
                                max(dentcas.p[,1],dentcon.p[,1])+1.5*casbandw),
                             c( min(dentcas.p[,2],dentcon.p[,2])-1.5*casbandw,
                                max(dentcas.p[,2],dentcon.p[,2])+1.5*casbandw) ) )

# 'inside' a boolean defined above (T/F)...this sets all values outside the polygon to NA
# which avoids some divide by Inf and NaN errors.

frand$fhat[!inside] <- NA
grand$fhat[!inside] <- NA

# This sets the RR value to the frand value (to preset all of the NAs in the right place)
rrand <- frand

# Reset the fhat value to the log relative risk value
rrand$fhat <- log(frand$fhat/grand$fhat)

# calculate test statistic (omit Inf's and NA's)

```



```

test.stat.sim[i] <- sum( (rrand$fhat[is.finite(rrand$fhat) & !is.na(rrand$fhat)]/

# assign row of z values to matrix of simulation values

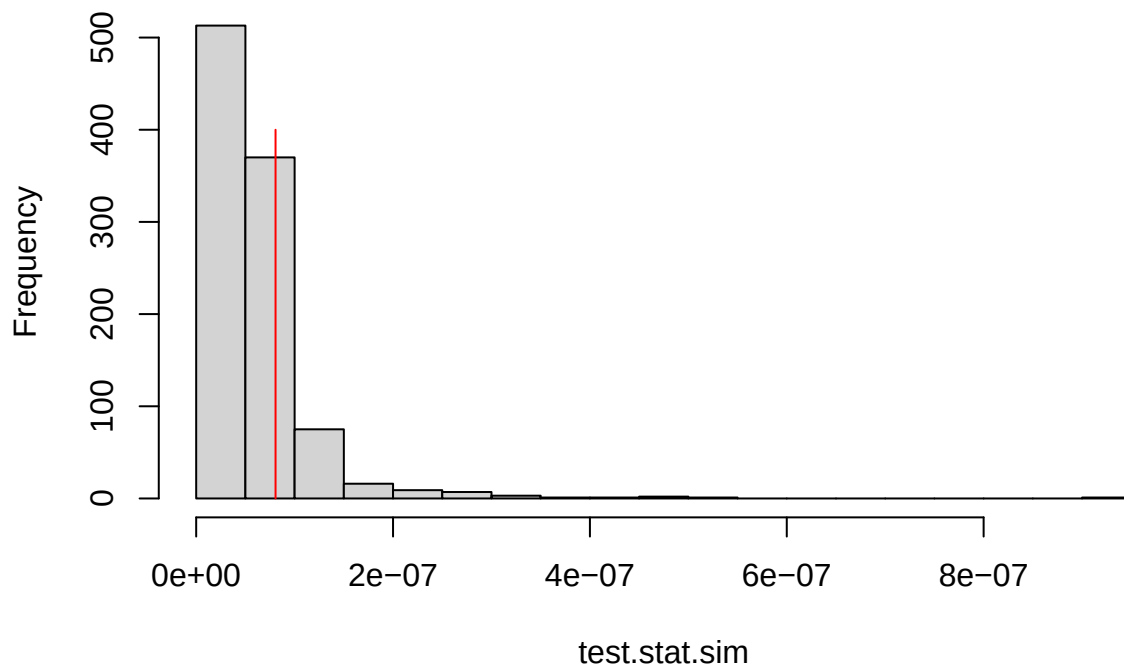
for (j in 1:grid.number) {
  rrandmat[i,((j-1)*grid.number+1):(j*grid.number)] <- rrand$fhat[j,]
}

} # end of sim loop

# Now make a histogram of the test statistics from the random labelling data
# and show the test statistic from the observed data.
#pdf("dental.KD700.hist.pdf",width=11,height=8)
par(mfrow=c(1,1),pty="m")
hist(test.stat.sim,nclass=25,
      main=paste("test stat = ",round(test.stat.obs,9)," , pval = ",
        round(length(test.stat.sim[test.stat.sim>test.stat.obs])/
          length(test.stat.sim),4)))
segments(test.stat.obs,0,test.stat.obs,400,col="red")

```

test stat = 8.1e-08 , pval = 0.1912



```

#dev.off()

#####
# Now to get the tolerance intervals for each grid point
#
# Find indices of non-NA elements 'nona' means it is not an NA
index.nona <- 1:(grid.number^2)

rz.vec <- r.ks[1,]

```

```

for (i in 2:grid.number) {
  rz.vec <- c(rz.vec,r.ks[i,])
}

index.nona <- index.nona[!is.na(rz.vec)]

#####
# Calc quantiles

# First set a matrix that will have the values in it.
rquantmat <- matrix(0,5,length(index.nona))

rrandmat[is.na(rrandmat)] <- -9999

for (j in 1:length(index.nona)) {
  rquantmat[,j] <- quantile(rrandmat[,index.nona[j]],
    c(0.0,0.025,0.5,0.975,1.0))
}

quant.05 <- rep(NA,grid.number*grid.number)
quant.50 <- rep(NA,grid.number*grid.number)
quant.95 <- rep(NA,grid.number*grid.number)

quant.05[index.nona] <- rquantmat[2,]
quant.50[index.nona] <- rquantmat[3,]
quant.95[index.nona] <- rquantmat[4,]

quant.05.mat <- matrix(quant.05,grid.number,grid.number)
quant.50.mat <- matrix(quant.50,grid.number,grid.number)
quant.95.mat <- matrix(quant.95,grid.number,grid.number)

# set -9999's to NA's (so they don't map)
quant.05.mat[quant.05.mat== -9999] <- NA

#contour(r$x,r$y,quant.05.mat)
#contour(r$x,r$y,quant.50.mat)
#contour(r$x,r$y,quant.95.mat)

#persp(r$x,r$y,quant.05.mat,theta=30,phi=45)
#persp(r$x,r$y,quant.50.mat,theta=30,phi=45)
#persp(r$x,r$y,quant.95.mat,theta=30,phi=45)

```

Now plot (map) the results!

```

#pdf("dentalKD.tol700.KS.pdf",width=11,height=8)
par(mfrow=c(1,2),pty="s")

persp(casdens$x1,casdens$x2,r.ks,theta=-25,phi=45,
  xlab="u",ylab="v",zlab="Log relative risk",zlim=c(-3,3))
title(paste("Log relative risk surface"),cex.main=1.0)

```

```

plot(dentcon.p,pch=" ",xlim=c(min(dental$x)-extra,max(dental$x)+extra),
     ylim=range(dental$y),xlab="u",ylab="v",cex.axis=1.25,cex.lab=1.5)
contour(casdens$x1,casdens$x2,r.ks,levels=c(-2,-1.5,-1,-0.5,0,0.5,1,1.5,2),
        lwd=c(1,1,1,1,2,3,3,3,3),
        lty=c(1,1,1,1,2,1,1,1,1),
        vfont=c("sans serif","bold"),
        labcex=1.25,add=T )
polygon(mypoly)
#   points(dentcas.p,pch=16)
#   points(dentcon.p,pch=1)
title(paste("Gaussian kernel, Bandwidth = ",bandw),cex.main=1.0)

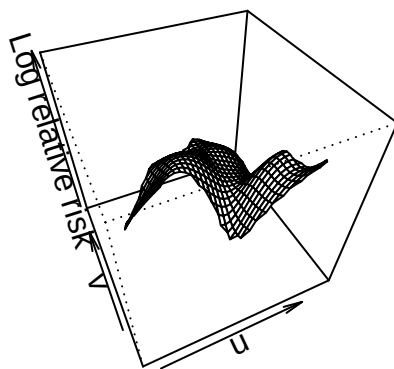
# Create long vectors of (x,y) coords for r

rx <- rep(casdens$x1,length(casdens$x2))
for (i in 1:length(casdens$x2)) {
  if (i == 1) {
    ry <- rep(casdens$x2[i],length(casdens$x1))
  }
  if (i != 1) {
    ry <- c(ry,rep(casdens$x2[i],length(casdens$x1)) )
  }
}

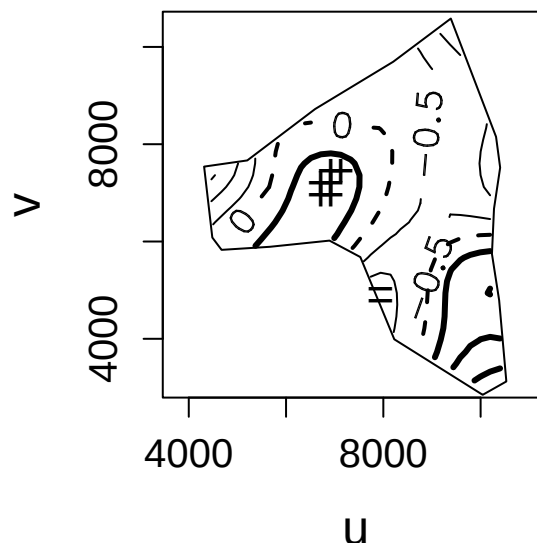
points(rx[r.ks>quant.95.mat],ry[r.ks>quant.95.mat],pch="+",cex=1.5)
points(rx[r.ks<quant.05.mat],ry[r.ks<quant.05.mat],pch="-",cex=1.5)

```

Log relative risk surface



Gaussian kernel, Bandwidth = 700



```
#dev, off()
```

Since this example doesn't have strong clustering, only a few symbols are added and these vary slightly from run-to-run (since these are based on Monte Carlo simulations). Look for a couple of “-” symbols along the

lower left edge that often show up.

Now let's try this with the 'sparr' (spatial relative risk) R function from

T. M. Davies, J. C. Marshall, M. L. Hazelton (2018). Tutorial on kernel estimation of continuous spatial and spatiotemporal relative risk; *Statistics in Medicine*, 37(7), 1191-1221.

<https://tilmandavies.r-universe.dev/sparr>

```
#load the package
library(spatial)
library(sparr)

## Loading required package: spatstat
## Loading required package: spatstat.data
## Loading required package: spatstat.univar
## spatstat.univar 3.2-0
## Loading required package: spatstat.geom
## spatstat.geom 3.8-1
## Loading required package: spatstat.random
## spatstat.random 3.5-0
## Loading required package: spatstat.explore
## Loading required package: nlme
## spatstat.explore 3.8-1
## Loading required package: spatstat.model
## Loading required package: rpart
## spatstat.model 3.7-1
##
## Attaching package: 'spatstat.model'
## The following object is masked from 'package:spatial':
##
##   Strauss
## Loading required package: spatstat.linnet
## spatstat.linnet 3.5-1
##
## spatstat 3.6-1
## For an introduction to spatstat, type 'beginner'
##
##
## Welcome to
##
##  /---//  - \ / - \ / - \ / - \
##  \_  \ /  ---/  -  /  ---/  ---/
##  ---/  /  /  /  /  /  /  \  \  \  \
```

```

## /____/_/  /_/_/_/_/  \_/_/  \_\  v2.3-16
##
## - type news(package="sparr") for an overview
## - type help("sparr") for documentation
## - type citation("sparr") for how to cite

#define case and control locations
# First define a window around all of the points (the 'spatial' package
# defines a square window using the as.owin function).
xmin = min(c(dentcas.p[,1],dentcon.p[,1]))
xmax = max(c(dentcas.p[,1],dentcon.p[,1]))
ymin = min(c(dentcas.p[,2],dentcon.p[,2]))
ymax = max(c(dentcas.p[,2],dentcon.p[,2]))
Window = as.owin(c(xmin,xmax,ymin,ymax))

# Now, using this window, use 'as.ppp' to define a point process object.
# The ppp format for points is what 'sparr' and 'spatial' use.
dental.all = as.ppp(dental.p,W=Window)
case = as.ppp(dentcas.p,W=Window)
control = as.ppp(dentcon.p,W=Window)

#define the bandwidth
h0 = 700
# Compute a symmetric (pooled) adaptive relative risk estimate
# with tolerance contours
dental_rr_700 <- risk(bivariate.density(case,h0=h0), bivariate.density(control,h0=h0), h0=h0, adapt=TRUE,
                    hp=OS(dental.all)/2, pilot.symmetry="pooled", davies.baddeley=0.05)

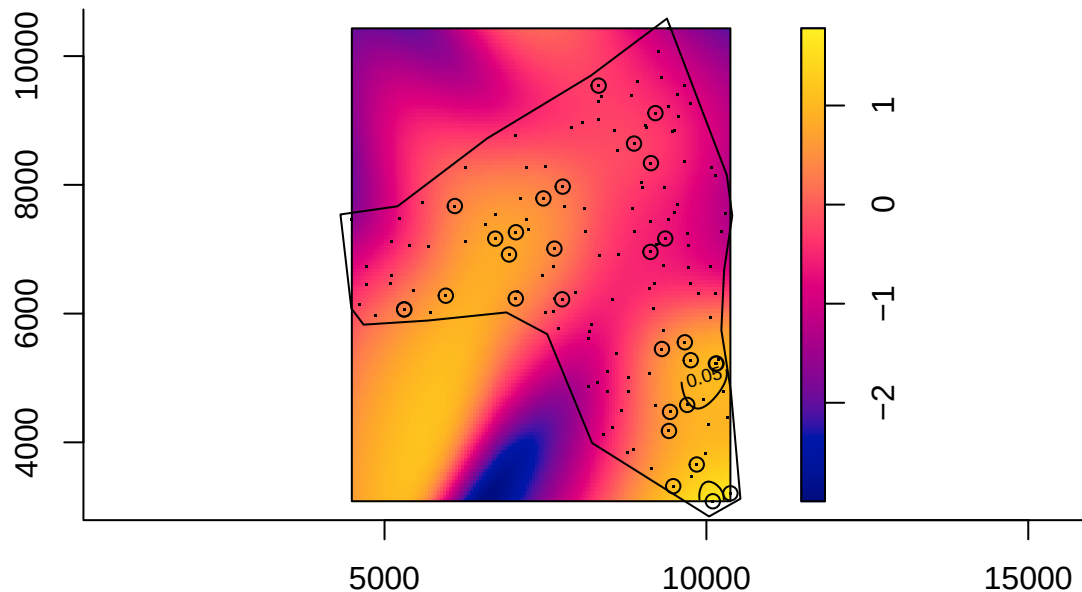
## Calculating tolerance contours...

## Done.

plot(dental_rr_700)
polygon(mypoly)
points(dental.all,pch=".")
points(case)
title("Sparr relative risk surface, bandwidth = 700")

```

Sparr relative risk surface, bandwidth = 700



This provides a heatmap-type contour plot, compare this to the contour plot above.

The sparr package also does Monte Carlo tests at each point in the grid and will put contours when the surface exceeds the 95% tolerance bands around the null value. These show up as contours (but can be a little hard to read).

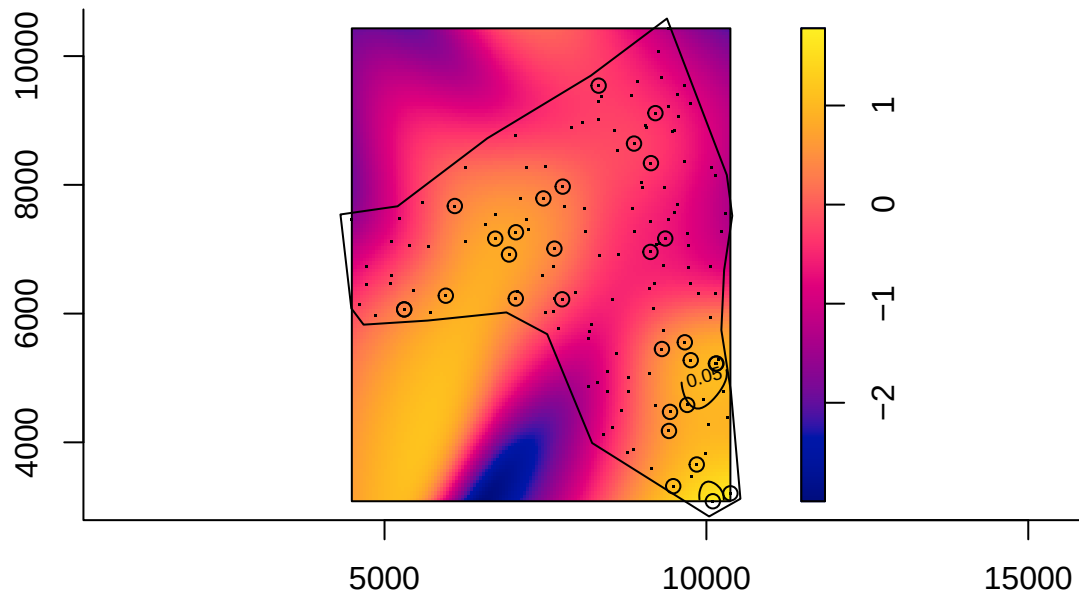
I've also put the polygon outline over the surface. We can see that the package does extend the surface into areas with no data, something we may not wish to do.

Let's try a different color scheme. We loaded the RColorBrewer package at the start of this file. Let's try the RColorBrewer blue-to-purple palette (which we used in the mapping of the Houston slippery slopes lab).

I'll repeat the default color scheme from the sparr package and show the same map with the RColorBrewer blue-to-purple scheme... which is easier to read, understand, explain?

```
plot(dental_rr_700)
polygon(mypoly)
points(dental.all, pch=".")
points(case)
title(paste("RR surface, bw = ", round(h0, digits=3), " default color"))
```

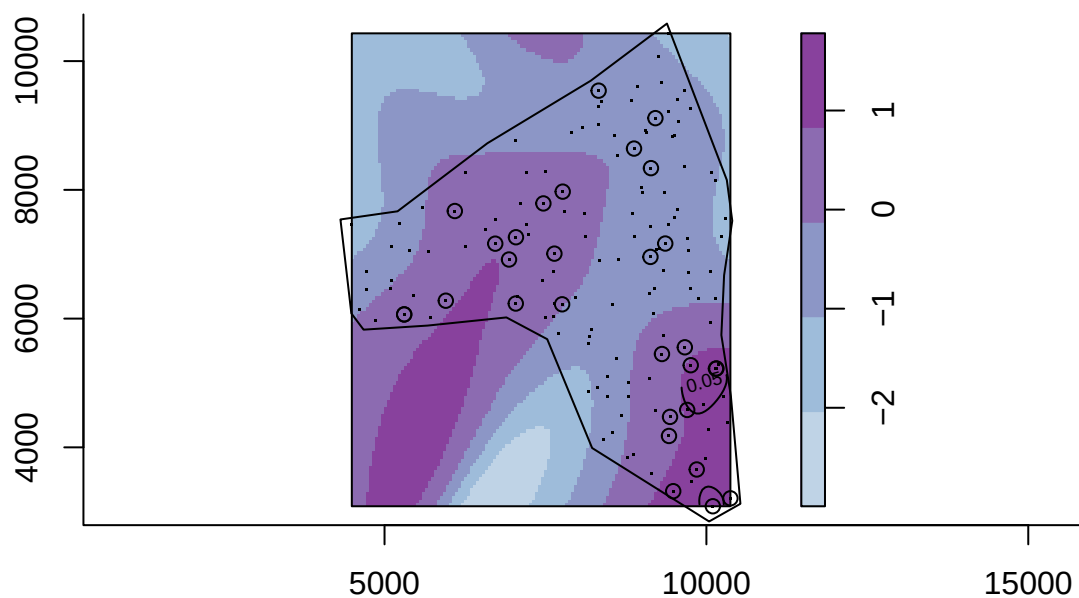
RR surface, bw = 700 default color



```
# set the color palette from RColorBrewer, and subset to start from third color
all_colors <- brewer.pal(9, "BuPu")
subset_colors <- all_colors[3:(1+6)]

plot(dental_rr_700,col=subset_colors)
polygon(mypoly)
points(dental.all,pch=".")
points(case)
title(paste("RR surface, bw = ",round(h0,digits=3), " Blue to Purple"))
```

RR surface, bw = 700 Blue to Purple



Now let's try with a smaller bandwidth (should yield a less smooth relative risk surface).

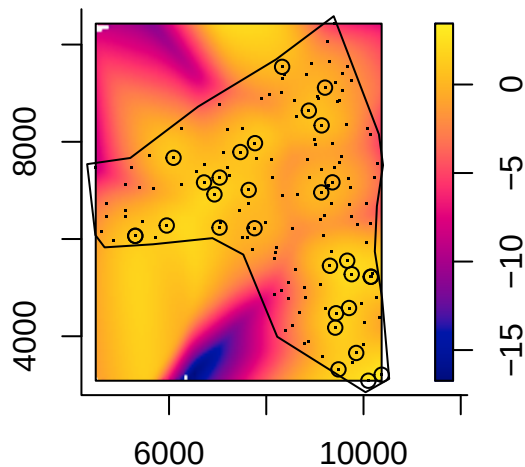
```
#define the bandwidth
h0 = 350
# Compute a symmetric (pooled) adaptive relative risk estimate with tolerance contours
dental_rr_350 <- risk(bivariate.density(case,h0=h0), bivariate.density(control,h0=h0),
                     h0=h0, adapt=TRUE, tolerate=TRUE, hp=OS(dental.all)/2,
                     pilot.symmetry="pooled", davies.baddeley=0.05)
```

```
## Calculating tolerance contours...Done.
```

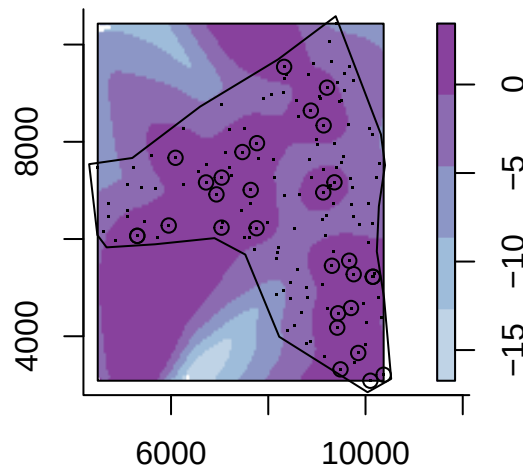
And we'll try both color schemes again.

```
# Let's set plotting parameters (par) so that the plot type (pty) is square (pty="s")
# and we have "multiple figures in a row" with 1 row and 2 figures (mfrow = c(1,2)).
par(pty="s",mfrow=c(1,2))
plot(dental_rr_350)
polygon(mypoly)
points(dental.all,pch=".")
points(case)
title(paste("RR, bw = ",round(h0,digits=2), " default color"))
plot(dental_rr_350,col=subset_colors)
polygon(mypoly)
points(dental.all,pch=".")
points(case)
title(paste("RR, bw = ",round(h0,digits=2), " Blue to Purple"))
```

RR, bw = 350 default color



RR, bw = 350 Blue to Purple



Next, we can mirror sparr's example and automatically calculate the bandwidth meeting thier optimality criteria as follows (this mirrors their example on their GitHub site). Pretty close to our bandwidth of 700 above.

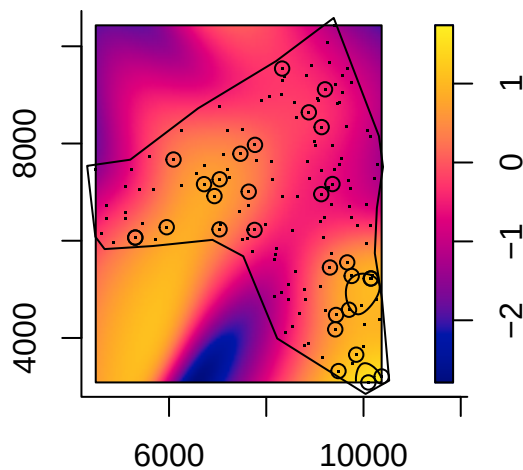
```
h0 <- OS(dental.all, nstar="npoints")

# Compute a symmetric (pooled) adaptive relative risk estimate
# with tolerance contours
dental_rr_h0 <- risk(bivariate.density(case,h0=h0), bivariate.density(control,h0=h0), h0=h0, adapt=TRUE,
                    hp=OS(dental.all)/2, pilot.symmetry="pooled", davies.baddeley=0.05)

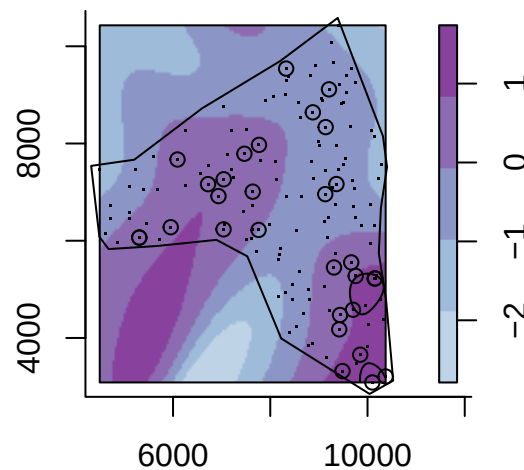
## Calculating tolerance contours...Done.

par(pty="s",mfrow=c(1,2))
plot(dental_rr_h0)
polygon(mypoly)
points(dental.all,pch=".")
points(case)
title(paste("RR, bw = ",round(h0,digits=1), " default color"))
plot(dental_rr_h0,col=subset_colors)
polygon(mypoly)
points(dental.all,pch=".")
points(case)
title(paste("RR, bw = ",round(h0,digits=1), " Blue to Purple"))
```

RR, bw = 718.1 default color



RR, bw = 718.1 Blue to Purple



The sparr package comes with some example data sets including 761 cases of primary biliary cirrhosis and 3,020 controls representing the at-risk population for 1987-1994. Data from:

Prince et al. (2001), The geographical distribution of primary biliary cirrhosis in a well-defined cohort, Hepatology, 34, 1083-1088.

and accessed from the sparr website <https://tilmandavies.r-universe.dev/sparr>

Here is the code from the Tilman Davies' sparr site.

```
### ALL CODE IN THIS CHUNK FROM TILMAN DAVIES website:
#### https://tilmandavies.r-universe.dev/sparr

# Load data on cases of primary biliary cirrhosis from north east England
data(pbc)

# Split into cases and controls
pbc_case <- split(pbc)$case
pbc_cont <- split(pbc)$control

# Estimate global bandwidth for smoothing
h0 <- OS(pbc, nstar="geometric")

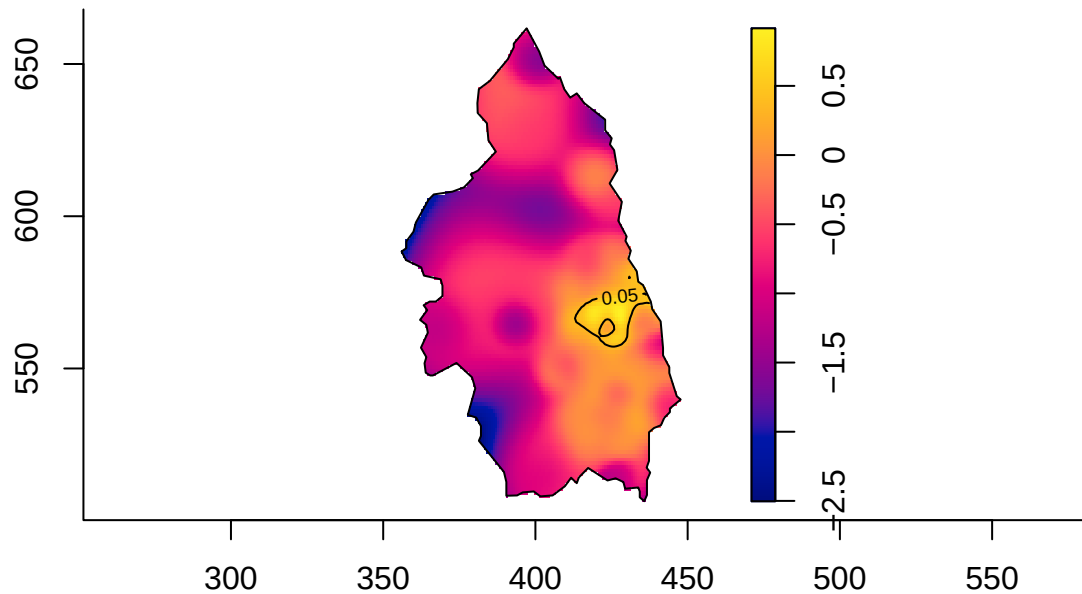
# Compute a symmetric (pooled) adaptive relative risk estimate
# with tolerance contours
pbc_rr <- risk(pbc_case, pbc_cont, h0=h0, adapt=TRUE, tolerate=TRUE,
              hp=OS(pbc)/2, pilot.symmetry="pooled", davies.baddeley=0.05)

## Estimating case density...Done.
## Estimating control density...Done.
## Calculating tolerance contours...Done.

#> Estimating case density...
#> Done.
#> Estimating control density...Done.
#> Calculating tolerance contours...Done.

# And produce a plot
plot(pbc_rr)
title(paste("RR surface, bw = ",round(h0,digits=3), " default color"))
```

RR surface, bw = 3.498 default color

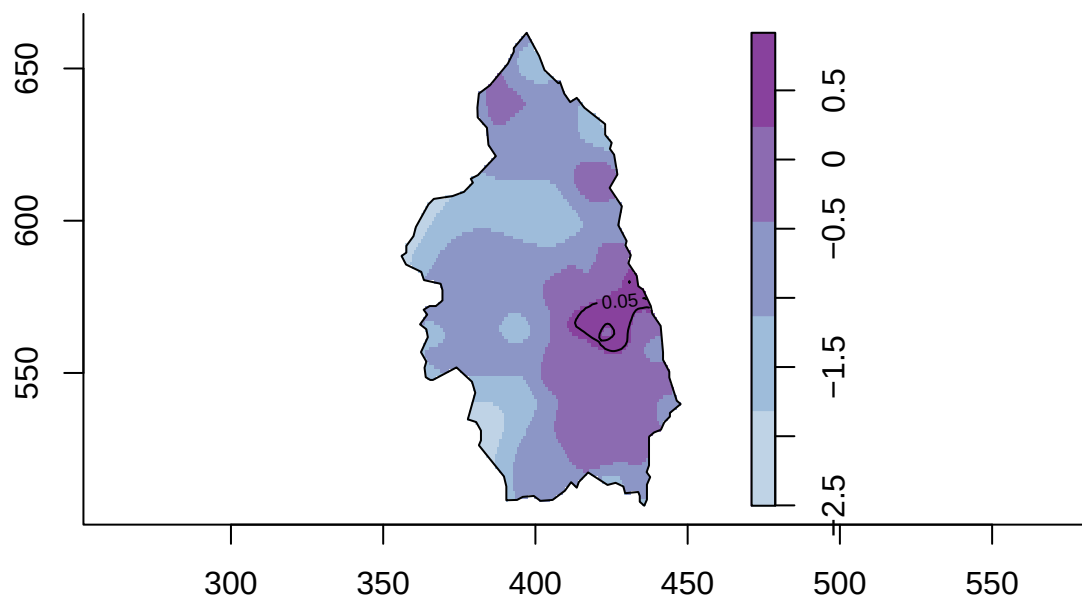


Let's change the color palette using a palette from the RColorBrewer package as above.

```
all_colors <- brewer.pal(9, "BuPu")
# Subset to start from the third color
subset_colors <- all_colors[3:(1+6)]
#Display the colors
#subset_colors

plot(pbc_rr,col=subset_colors)
title(paste("RR surface, bw = ",round(h0,digits=3), " Blue to Purple"))
```

RR surface, bw = 3.498 Blue to Purple



And make it a square plot and add the points

```
par(pty="s")  
plot(pbc_rr,col=subset_colors)  
points(pbc,pch=".",col="grey25")  
points(pbc_case,pch=".",col="red")
```

